

Publication and Performance

Check assets, metadata, source attribution and loading decisions before publishing a preview.



Learning outcomes

By the end of this lecture you can audit a page and justify that it is ready to publish.

Audit

Inspect every asset a page ships — images, fonts, scripts — and identify what is unoptimised.

Describe

Write correct metadata, Open Graph tags and source attribution so the page describes itself accurately.

Measure

Read Core Web Vitals and a Lighthouse report, and act on what they say before publishing.

“Done” is not the same as “ready to publish”

A page can render correctly on your machine and still fail its first real visitors.

Weeks 1–10 built structure, style, interaction, accessibility and tested revisions. This week adds the final gate: does the page perform, describe itself correctly, and credit its sources once it leaves your laptop?

What “done” checks

Content is correct, layout works, links go where expected — all judged on your own fast machine and fast connection.

What “ready to publish” adds

File sizes, metadata, attribution and load performance on a real network and a real device — conditions you cannot see from your editor.

The real asset audit

Every image, font, script and embed a page loads has a cost — list them before you ship.

Images	Often the largest share of page weight — check every image is sized and compressed for its actual display size.
Fonts	Each font file is a network request that can delay visible text — audit how many weights and styles are actually used.
Scripts	Third-party embeds (analytics, widgets, chat) each add their own request chain and can block rendering.
External embeds	Maps, videos and social embeds often load their own heavy sub-resources you did not directly write.

Image formats: choosing the right one

JPEG, PNG, WebP and AVIF trade off compression, transparency and browser support differently.

JPEG

Best for photographs: strong lossy compression, no transparency, universal support.

PNG

Best for graphics needing transparency or flat colour — much larger files for photographic content.

WebP

Smaller than JPEG/PNG at similar quality, supports transparency, supported by all modern browsers.

AVIF

Often the smallest of all at equal quality, but slower to encode and still gaining full tooling support.

Responsive images, revisited

Week 6 introduced srcset/sizes — this week, verify it is actually wired correctly before publishing.

```

```

A publish-ready image also states width and height attributes — the browser reserves the correct space before the file even downloads, preventing layout shift.

Compressing images without visible quality loss

A correctly-sized image can still be needlessly heavy if it isn't compressed.

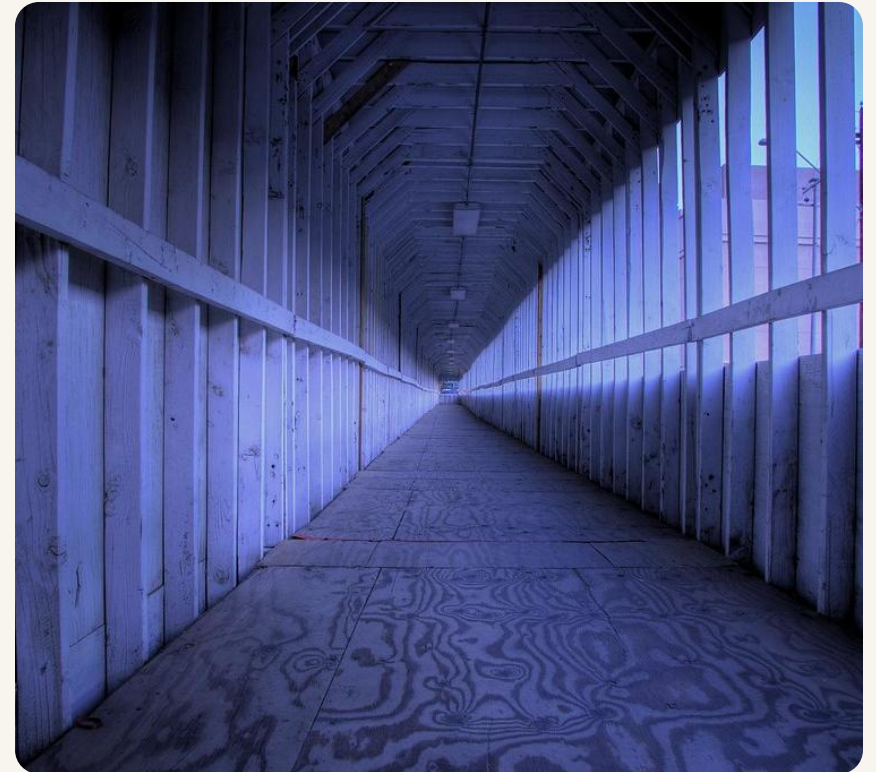
Re-encoding a photo at roughly 70–80% JPEG quality is usually visually indistinguishable from the original, but can cut file size by half or more.

Practical rule

When you upload an image to a website, make sure it's the right size and format for the web.

Common mistake

Uploading a large image to a website without compressing it first.



Loading strategy: eager, lazy and priority

Not every image needs to load immediately — but the first one usually does.

```
<!-- Hero image: load immediately, it's the first thing seen -->  
  
  
<!-- Below-the-fold image: defer until near viewport -->  

```

loading="lazy" on off-screen images and fetchpriority="high" on the hero image both target the same goal: get the content the visitor sees first onto the screen first.

Web fonts: display, subsetting, preloading

A font file that blocks text from appearing is one of the most common publish-day surprises.

```
<link rel="preload" href="/fonts/body.woff2"
      as="font" type="font/woff2" crossorigin>

@font-face {
  font-family: "Body";
  src: url("/fonts/body.woff2") format("woff2");
  font-display: swap;
}
```

font-display: swap shows fallback text immediately and swaps in the web font once it loads, instead of leaving text invisible while the font downloads.

Minification and removing dead code

Whitespace, comments and unused CSS/JS all cost bytes on every single page load.

Minification

Build tools strip whitespace, comments and shorten variable names in CSS/JS — no behaviour change, smaller files.

Dead code

Unused CSS rules and unused JavaScript functions still cost download and parse time even if they never run.

Astro's build step already minifies output automatically — the audit here is checking you have not hand-pasted a whole unused library or framework into a page for one small feature.

Page metadata: title and description

The two lines a search result or shared link actually shows first.

```
<title>Portfolio site — Jordan Lee, web design</title>  
<meta name="description"  
      content="A responsive, accessible portfolio built  
              for SLOP1512, showcasing three  
              interactive case studies.">
```

Each page needs its own distinct title and description — a template placeholder like “Untitled” or a copy-pasted description on every page is a genuine publish-readiness failure.

Open Graph tags: what a shared link shows

When a URL is pasted into chat or social media, these tags control the preview card.

```
<meta property="og:title" content="Portfolio site — Jordan Lee">
<meta property="og:description" content="A responsive, accessible portfolio.">
<meta property="og:image" content="https://example.edu/og-cover.jpg">
<meta property="og:url" content="https://example.edu/">
```

Without an `og:image`, most platforms show a blank or generic preview card — a working link that still looks broken the moment someone shares it.

Favicons and touch icons

A small, unglamorous checklist that is easy to forget entirely.

`favicon.ico` / `favicon.svg`

shown in the browser tab and bookmarks

`apple-touch-icon.png` (180×180)

used when a visitor adds the page to an iOS home screen

`site.webmanifest`

names and icons the page if added to an Android home screen

Why source attribution matters

Every photograph, icon or quote used in this course's decks and sites is someone else's work.

A Creative Commons or public-domain licence permits reuse, but almost all of them still require crediting the creator — using an image without attribution can violate the very licence that made it usable.

Minimum credit

Academic parallel



A working attribution pattern

Record it once, in a predictable place, at the time you download the asset — not from memory later.

```
{  
  "file": "hero.jpg",  
  "title": "Team reviewing a printed layout",  
  "creator": "Jane Doe",  
  "license": "CC BY 2.0",  
  "source": "https://example.com/photos/12345",  
  "accessed": "2026-09-13"  
}
```

Why this shape

Every field needed to satisfy a CC BY licence's attribution requirement is present, and the file lives alongside the assets it describes so it cannot go missing.

Reading a Creative Commons licence correctly

Not all CC licences permit the same reuse — read the actual letters, not just the icon.

CC0 / Public domain

no attribution legally required (crediting anyway is still good practice)

CC BY

reuse and even modify freely, but attribution is required

CC BY-SA

attribution required, and anything you build with it must share alike under the same licence



Core Web Vitals: the three metrics that matter

Google's real-user metrics for loading, responsiveness and visual stability.

LCP

Largest Contentful Paint: how long until the biggest visible element (often a hero image) finishes rendering. Target: under 2.5s.

INP

Interaction to Next Paint: how quickly the page responds to a click, tap or keypress. Target: under 200ms.

CLS

Cumulative Layout Shift: how much visible content jumps around as the page loads. Target: under 0.1.

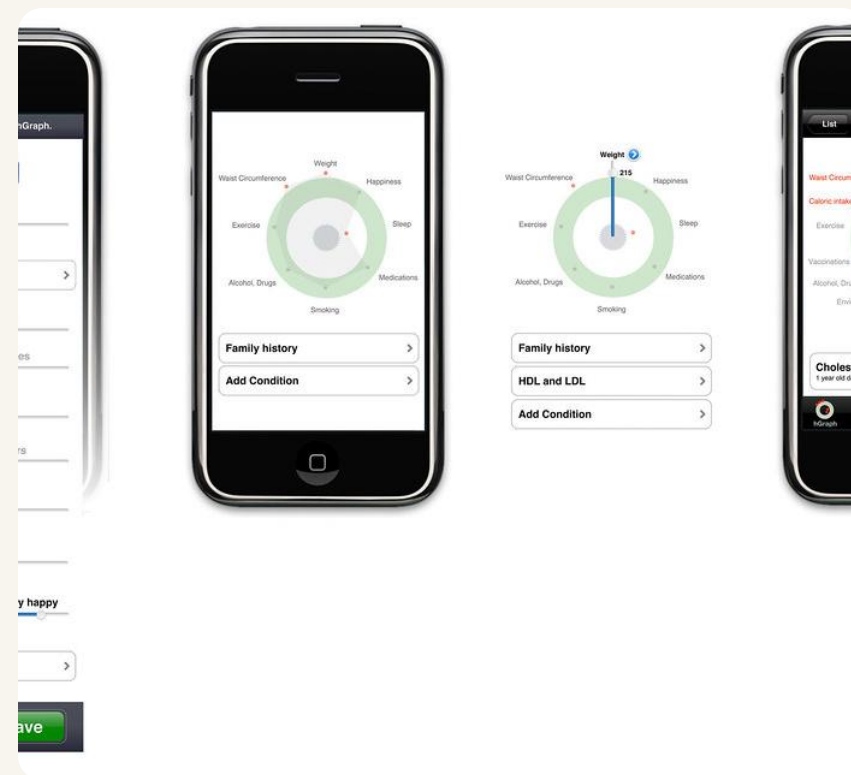
Measuring LCP with real tooling

Do not guess — read what the browser itself measured.

Chrome DevTools' Performance panel and the Lighthouse panel both report a measured LCP time and identify exactly which element on the page was counted as the largest.

What to check

Field vs lab data



Preventing layout shift

CLS is almost always caused by content whose final size isn't known until it loads.

```
img, video, iframe {
  aspect-ratio: attr(width) / attr(height);
}

.ad-slot {
  min-height: 250px; /* reserve space before the ad loads */
}
```

Explicit width/height on images (Week 6) and aspect-ratio reservation on embeds are the two most common fixes for a visible page-load jump.

Testing on a throttled connection

Your dev machine's wifi is not representative of a real visitor's phone on the move.

DevTools' network throttling presets (“Slow 4G”, “Fast 3G”) simulate realistic latency and bandwidth, revealing loading problems a fast office connection completely hides.

What changes

CPU throttling too



From files to a live URL: what hosting does

Publishing is not just “upload the folder” — a host serves, compresses and caches your files.

A static host (like the one this course's platform deploys to) serves pre-built files directly, applies HTTP compression (gzip/brotli) and sets caching headers so returning visitors re-download less.

Check before publishing

Cache-Control headers



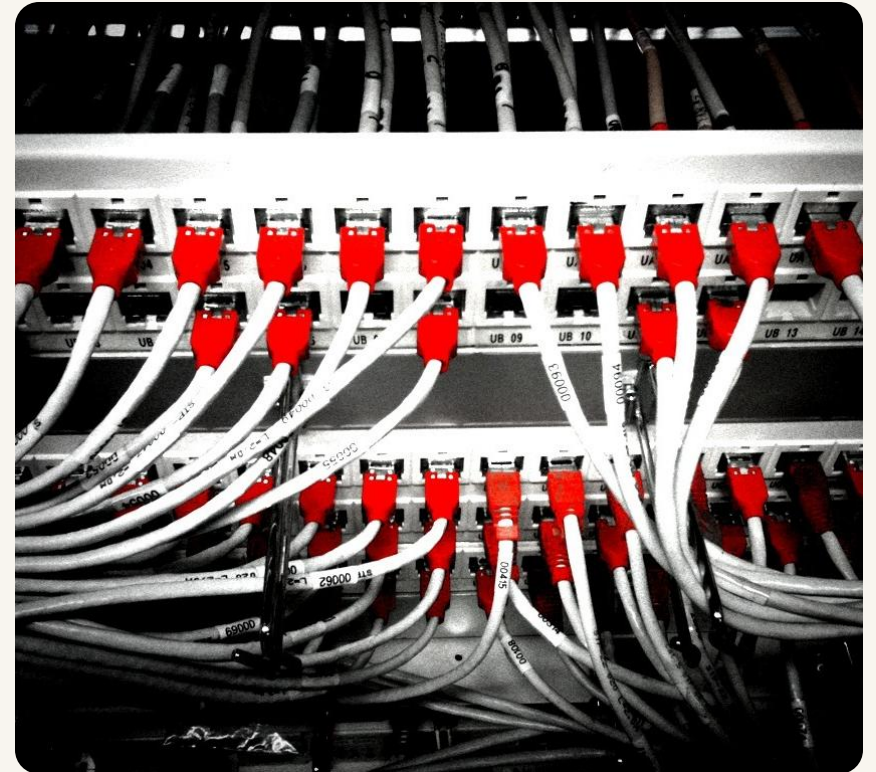
CDNs and DNS: how bytes travel to a browser

A content delivery network puts copies of your files physically closer to each visitor.

Instead of every visitor's request travelling to one origin server, a CDN caches your published files at edge locations worldwide, cutting the physical distance data has to travel.

Why this matters here

DNS resolution



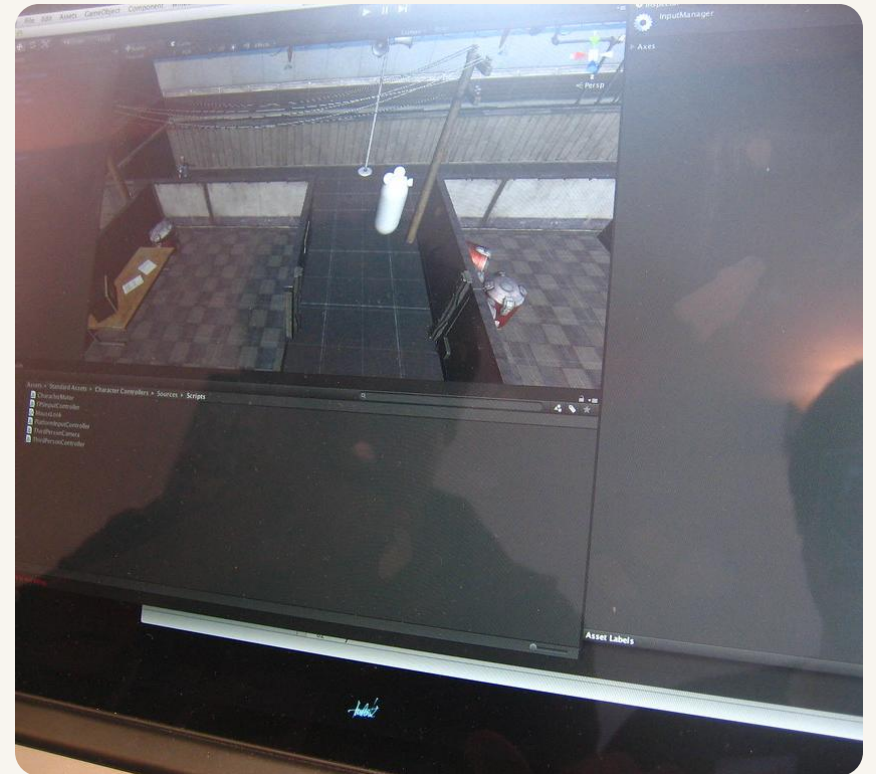
Reading a Lighthouse / PageSpeed Insights report

One number is a summary; the audit list underneath is where the real work is.

The overall performance score (0–100) is useful for tracking trend over time, but the individual audits — “serve images in next-gen formats,” “eliminate render-blocking resources” — are the actionable checklist.

Prioritise by impact

Re-run after every fix



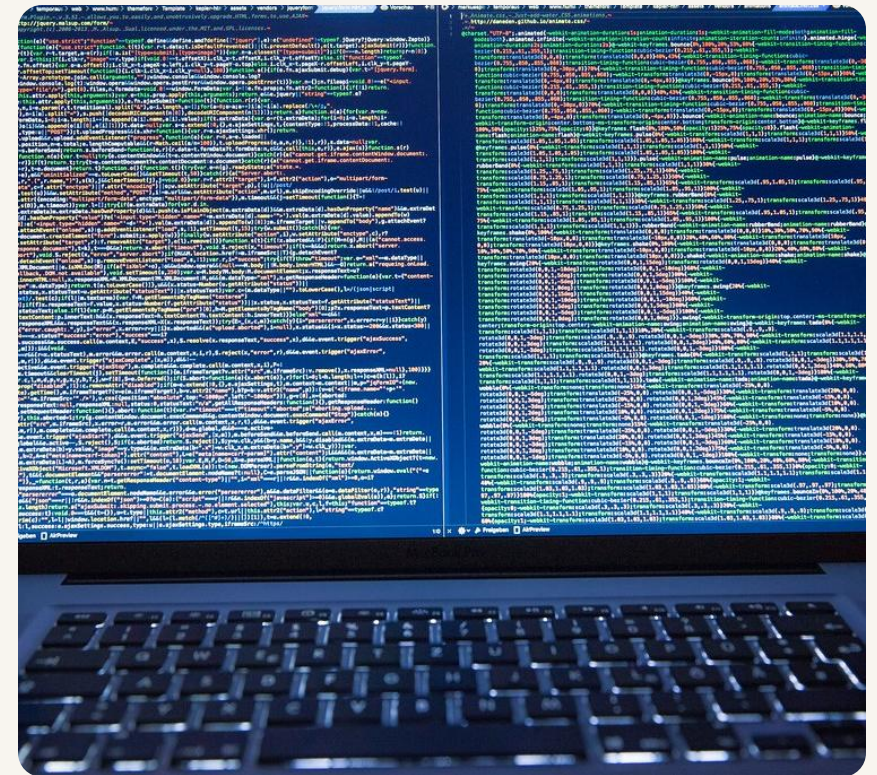
Case study, before: an unaudited page

A real composite example — what the page looked like before this week's pass.

The original page shipped four camera-original JPEGs (avg. 3.2MB each), no width/height attributes, no meta description, and a font loaded with no font-display rule.

Measured before

Root cause



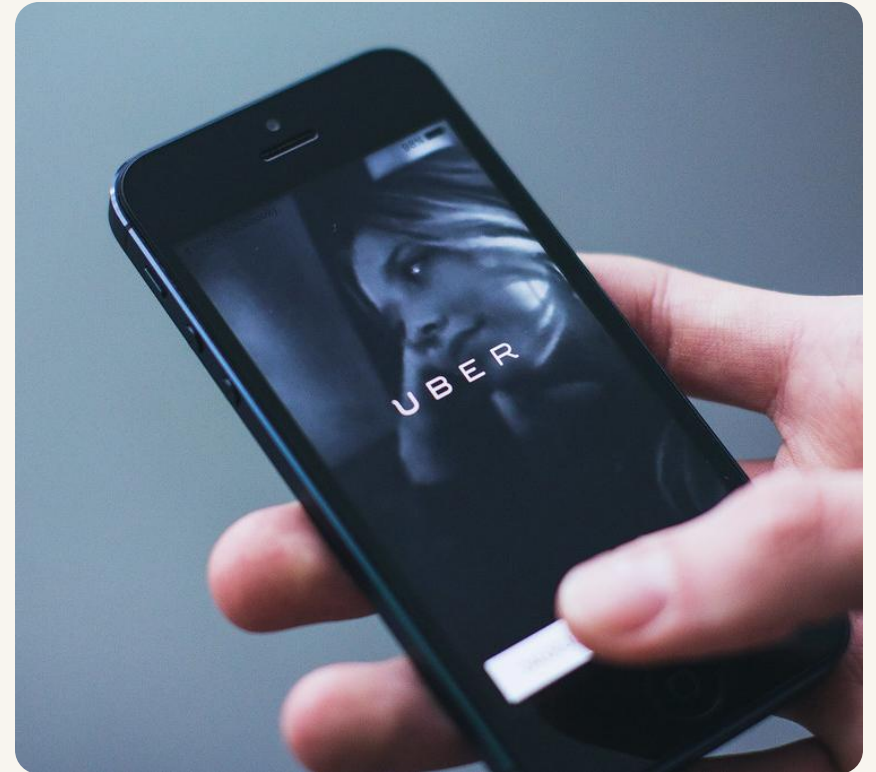
Case study, after: the Week 11 pass applied

Same content, now measurably ready to publish.

Images resized and compressed to WebP (avg. 180KB), width/height attributes added, a written meta description and og:image added, font-display: swap applied.

Measured after

Takeaway



Pre-publish checklist

Run this list before every submission or public deploy this course asks for.

Assets

Every image sized, compressed and given width/height.

Metadata

Unique title, meta description, Open Graph and favicon set.

Performance

LCP, INP and CLS measured with Lighthouse under throttled network and CPU conditions, and the top 2–3 audits acted on before submitting.

Attribution

Every reused image, icon or quote credited with licence and source link.



Studio task

Audit and fix one page from an earlier week's work.

Brief

Take a page you built earlier in this course. Run Lighthouse, record the LCP/INP/CLS scores, then fix the two highest-impact audits it lists.

Constraints

Add a correct title, meta description and at least one Open Graph tag. Add an `ATtribution.json` entry for every non-original image on the page.

Evidence to submit

Before/after Lighthouse scores, the two audits you fixed and how, and your completed `ATtribution.json` for that page.

Review checklist

How publication-readiness is assessed in project review.

Assets are optimised

Images correctly sized, compressed and given width/height; unused code removed.

Metadata is accurate

Unique title, meta description and working Open Graph tags for every page.

Sources are credited

Every reused image, icon or quote has a recorded creator, licence and link.

Core Web Vitals measured

LCP, INP and CLS reported from a throttled test, not just a fast local connection.

Connections to earlier weeks

Publication and performance closes the loop on decisions made since Week 1.

Week 6 — responsive images

srcset/sizes and width/height attributes from Week 6 are exactly what CLS and LCP measure here.

Week 8 — accessibility review

Alt text audited in Week 8 lives in the same asset-audit pass as file size and format checked this week.

Week 10 — user testing

A blocker found by a real user is worthless if the page that fixes it never actually loads for them in time.

Week 12 (next)

The portfolio review presents the finished, published site — this week is the last gate before that becomes public.

Next week

Week 12: Portfolio review.

Present the final site, explain the design system behind it, and reflect on the testing and revision evidence gathered across the course — including the publish-readiness pass completed this week.